

UNIVERSITATEA “ȘTEFAN CEL MARE”, SUCEAVA
FACULTATEA DE INGINERIE ELECTRICĂ ȘI ȘTIINȚA
CALCULATOARELOR
SPECIALIZAREA CALCULATOARE
AN 2, GRUPA 3123B

Proiect POO: Motor de Căutare Documente Text

Profesor coordonator: PRODAN Remus-Cătălin

Student: Nichitean Alberto | Grupa: 3123B

Analiza Temei

Tema presupune realizarea unui sistem capabil să indexeze documente text și să permită căutarea eficientă a cuvintelor. Am ales o abordare modulară care să permită extinderea ulterioară a tipurilor de documente suportate (ex: PDF sau HTML). Pentru a optimiza căutarea, sistemul include un filtru anti-plagiat/stop-words, iar interacțiunea utilizatorului este jurnalizată printr-un sistem decuplat de log-uri.

Clasele și Interfețele Aplicației

- **IDocument (Interfață):** Clasa pur abstractă care definește comportamentul obligatoriu pentru orice document (încărcare conținut, obținere cale).
- **TextDocument:** Implementarea concretă pentru fișiere de tip text, care moștenește interfața IDocument.
- **InvertedIndex:** „Creierul” aplicației. Stochează un `std::map` ce mapează fiecare cuvânt către lista de documente asociate. Gestionează ignorarea cuvintelor de legătură (stop-words) și rezolvă interogările logice.
- **IndexException:** O clasă proprie de erori, creată pentru a raporta probleme la nivelul fișierelor sau directoarelor (ex: folder inexistent).
- **IObserver & Logger:** Interfața abstractă și clasa concretă utilizate pentru implementarea tiparului Observer, având rolul de a monitoriza automat sistemul.

Concepte POO Utilizate

- **Abstractizare:** Realizată prin utilizarea claselor abstracte (IDocument, IObserver) pentru a defini "contracte" clare de implementare.
- **Moștenire:** TextDocument derivă din IDocument. IndexException derivă din clasa de bază `std::exception`. Logger derivă din IObserver.
- **Încapsulare:** Atributele claselor sunt declarate private pentru a proteja integritatea datelor. Metodele ajutătoare sunt ascunse față de exterior.
- **Polimorfism Dinamic:** Utilizat prin metode virtuale. Clasa InvertedIndex procesează pointeri de tip `IDocument*` și `IObserver*`, permițând tratarea uniformă a obiectelor derivate la runtime.

- Tratarea Excepțiilor: Aplicată prin folosirea blocurilor try-catch pentru a intercepta obiecte de tip `IndexException` curat, prevenind oprirea necontrolată a programului.
- Design Pattern (Observer): Implementat pentru a decupla total logica de căutare de logica de jurnalizare. Motorul de căutare nu știe nimic despre Logger; el doar își notifică lista globală de observatori, sporind extensibilitatea aplicației.

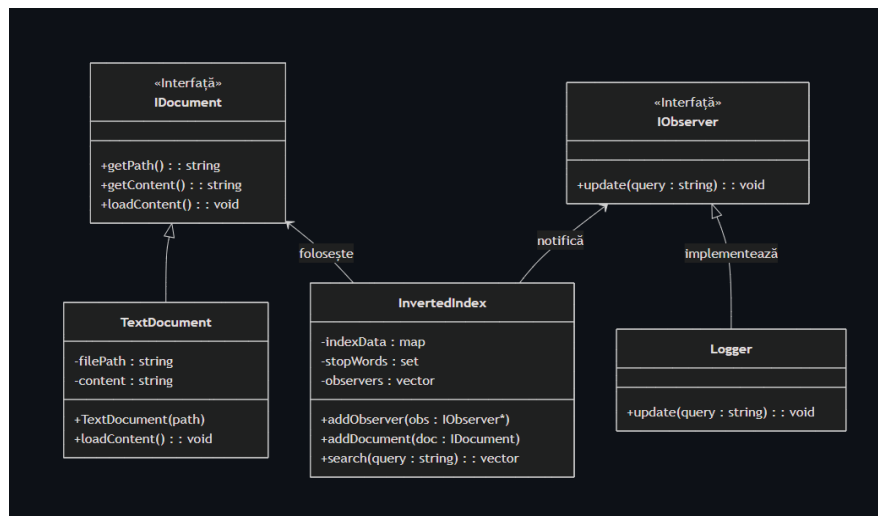
Funcționalități Tehnice Avansate

- Citire Dinamică: Sistemul utilizează biblioteca standard `std::filesystem` (C++17) pentru a scana automat directorul țintă și a încărca fișierele valide în memorie.
- Căutare Rapidă și Case-Insensitive: Căutările folosesc arborele binar din spatele `std::map`, oferind un timp de acces logaritm $O(\log n)$. Termenii sunt normalizați complet.
- Procesare Booleană (AND / OR): Algoritmul de căutare rezolvă intersecții (pentru clauza AND - returnând doar documentele cu toate cuvintele cerute) și reuniuni (pentru clauza OR).

Testare și Validare

- Teste Unitare: Logica motorului de căutare este validată independent folosind macroul `assert()`, demonstrând gestionarea corectă a cuvintelor valide, a operațiilor booleene, a stop-words-urilor și a termenilor inexistenți.

Diagrama UML a Sistemului



Concluzii și Posibile Îmbunătățiri

Proiectul și-a atins cu succes scopul, rezultând într-un motor de căutare robust, modular și extrem de rapid datorită utilizării structurilor de date potrivite (Inverted Index cu `std::map`). Respectarea principiilor POO asigură un cod ușor de întreținut.

Pentru dezvoltările ulterioare (scalare), aplicația ar putea fi îmbunătățită prin:

- **Extinderea tipurilor de fișiere:** Crearea unor noi clase derivate din **IDocument** (ex: **PdfDocument**, **HtmlDocument**) pentru a parse și alte formate, fără a modifica logica de bază.
- **Algoritm de Ranking (TF-IDF):** În loc ca interogările să returneze doar o listă simplă de documente, rezultatele ar putea fi ordonate în funcție de relevanță (de câte ori apare cuvântul în text comparativ cu restul corpusului).
- **Interfață Grafică (GUI):** Înlocuirea meniului din consolă cu o interfață vizuală folosind biblioteci precum Qt sau ImGui.
- **Multi-threading:** Indexarea concurentă a fișierelor atunci când directorul conține mii de documente, pentru a reduce timpul de pornire al aplicației.

Capturi de Ecran (Validarea Funcționalității)

Poza 1 (Meniul și Încărcarea):

```
Remote Console
+ Developer PowerShell
=== Motor de Cautare Documente (Faza 3 FINAL) ===

[INFO] Se pregatesc documentele din folderul 'documente_test'...
-> Gasit si pregatit: documente_test/istoric.txt
-> Gasit si pregatit: documente_test/programare.txt
-> Gasit si pregatit: documente_test/stiinta.txt

[INFO] Se incepe indexarea...
[INFO] Indexare completata cu succes!

=====
                MENU MOTOR DE CAUTARE
=====

1. Efectueaza o cautare (Simpla sau AND/OR)
2. Afiseaza documentele indexate in sistem
0. Iesire program
-----
Alege o optiune: █
```

Poza 2 (Filtrare Stop-Words / Fără rezultate):

```
Remote Console
+ Developer PowerShell
=====
                MENU MOTOR DE CAUTARE
=====

1. Efectueaza o cautare (Simpla sau AND/OR)
2. Afiseaza documentele indexate in sistem
0. Iesire program
-----
Alege o optiune: 1

-> Introdu un cuvnt (ex: 'teoria') sau o cautare avansata (ex: 'teoria AND fizica'):
Cauta: si

[LOGGER] A fost efectuată o cautare pentru: 'si'

[ REZULTATE ]
-> Interogarea 'si' nu a returnat niciun rezultat valid.
```

Poza 3 (Căutarea Avansată AND/OR):

```
Remote Console
+ Developer PowerShell | [Copy] [Paste] [Settings]

=====
                MENU MOTOR DE CAUTARE
=====
1. Efectueaza o cautare (Simpla sau AND/OR)
2. Afiseaza documentele indexate in sistem
0. Iesire program
-----
Alege o optiune: 1

-> Introdu un cuvânt (ex: 'teoria') sau o cautare avansata (ex: 'teoria AND fizica'):
Cauta: cod OR domnitor

[LOGGER] A fost efectuată o cautare pentru: 'cod OR domnitor'

[ REZULTATE ]
- Document: documente_test/programare.txt
- Document: documente_test/istoric.txt

Remote Console
+ Developer PowerShell | [Copy] [Paste] [Settings]

=====
                MENU MOTOR DE CAUTARE
=====
1. Efectueaza o cautare (Simpla sau AND/OR)
2. Afiseaza documentele indexate in sistem
0. Iesire program
-----
Alege o optiune: 1

-> Introdu un cuvânt (ex: 'teoria') sau o cautare avansata (ex: 'teoria AND fizica'):
Cauta: programator AND index

[LOGGER] A fost efectuată o cautare pentru: 'programator AND index'

[ REZULTATE ]
- Document: documente_test/programare.txt
```

Poza 4 (Afișareaentelor - Opțiunea 2):

```
Remote Console
+ Developer PowerShell
=====
                        MENU MOTOR DE CAUTARE
=====
1. Efectueaza o cautare (Simpla sau AND/OR)
2. Afiseaza documentele indexate in sistem
0. Iesire program
-----
Alege o optiune: 2

[ DOCUMENTE INCARCATE ]
- documente_test/istoric.txt
- documente_test/programare.txt
- documente_test/stiinta.txt
```

Poza 5 (Testele Unitare):

```
Remote Console
+ Developer PowerShell
[PASSED] Test 3: Cuvant inexistent tratat corect.
Toate testele au trecut cu succes!

Remote Console | Error List | Output
```